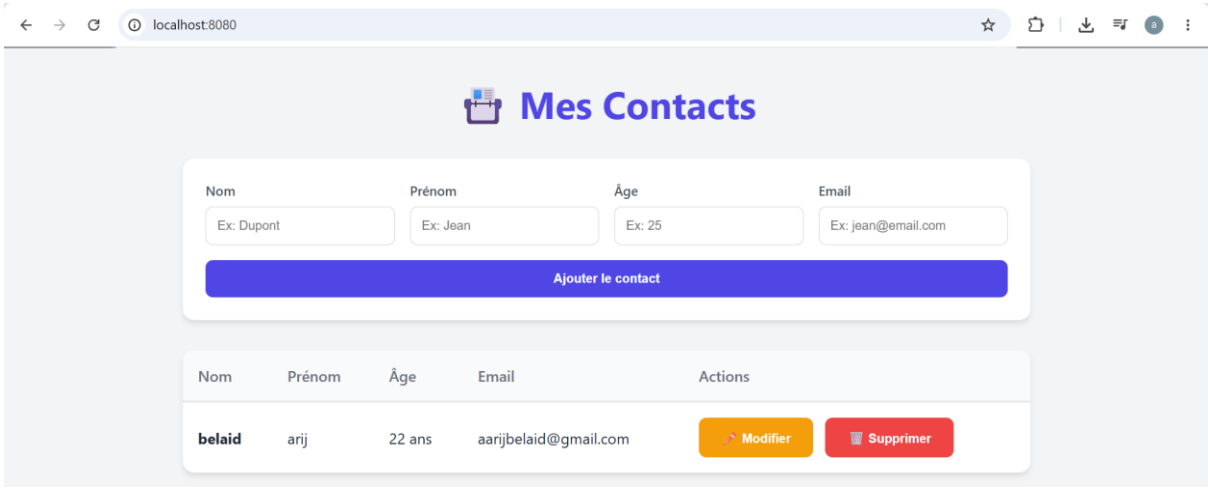
Partie 2 :Mini_Projet2

PHASE_0 : Développer et tester en local

1. Développer et Builder l'app:

```
PS C:\Users\arijb\Desktop\miniproj2> docker compose up --build -d
 => => exporting layers                                                                    0.2s
 => => exporting manifest sha256:53dd8bc7d726584a54b418a6e277bd4a42683e4fadcd479fdb5dfedcc1f6146b  0.0s
 => => exporting config sha256:2b5cb4309e4708a017619ca38273281a6a0c5eda6a5828a1e9c307272ce34a58  0.0s
 => => exporting attestation manifest sha256:76a0af87095b80cba91bca8ff5bacbb30d9e4abd9693ebc1ab49a5f23b6895a  0.1s
 => => exporting manifest list sha256:e761ea8b5902d2715aa73edb41291571b53f85bdd85f1d924487bd8480d1f80c  0.0s
 => => naming to docker.io/library/miniproj2-frontend:latest                               0.0s
 => => unpacking to docker.io/library/miniproj2-frontend:latest                            0.1s
 => [api 7/7] RUN chown -R appuser:appuser /app                                            0.7s
 => [api] exporting to image                                                               1.3s
 => => exporting layers                                                                    0.5s
 => => exporting manifest sha256:965c9883a1de88d7cf98c0b9ab59450bd7f70298242bd5e4e6dd074eb34ee8bf  0.0s
 => => exporting config sha256:6a8e9d662acc8266bc9bacf983690a3cc415abff43172a379d949b4edd07d7f7  0.1s
 => => exporting attestation manifest sha256:dea7893cebeab375424ebadbdd78e1ef267ebc4b2419a4bc46d16ef05e3b710  0.1s
 => => exporting manifest list sha256:938798a149f05b06d26a28e43f583a2ac6eb5c8e70528cf01fe7bf35d4e6820c  0.0s
 => => naming to docker.io/library/miniproj2-api:latest                                    0.0s
 => => unpacking to docker.io/library/miniproj2-api:latest                                 0.1s
 => [frontend] resolving provenance for metadata file                                      0.0s
 => [api] resolving provenance for metadata file                                           0.0s
[+] Running 5/5
 ✓ miniproj2-api                  Built                                                   0.0s
 ✓ miniproj2-frontend             Built                                                   0.0s
 ✓ Container miniproj2-db-1       Running                                                 0.0s
 ✓ Container miniproj2-api-1      Started                                                 4.8s
 ✓ Container miniproj2-frontend-1 Started                                                 3.7s
PS C:\Users\arijb\Desktop\miniproj2>
```

2.Teser :



PHASE_1 : Conteneuriser et publier :

1. tagger les images :

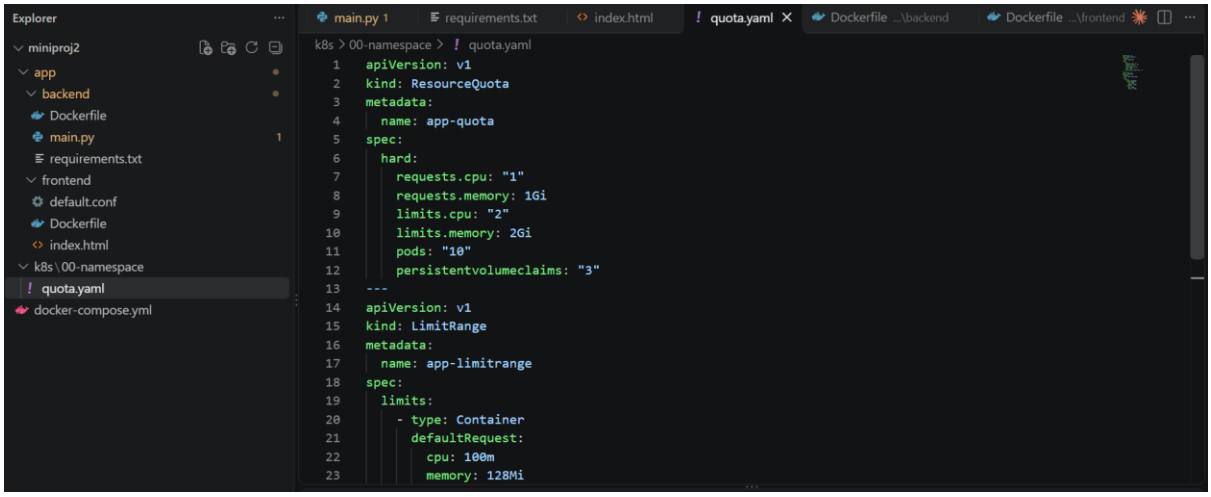
```
PS C:\Users\arijb\Desktop\miniproj2> docker tag miniproj2-api:latest arijbelaid/todo-api:1.0.0
>>
PS C:\Users\arijb\Desktop\miniproj2> docker tag miniproj2-frontend:latest arijbelaid/todo-frontend:1.0.0
>>
PS C:\Users\arijb\Desktop\miniproj2>
```

2.pusher sur docker hub

```
PS C:\Users\arijb\Desktop\miniproj2> docker push arijbelaid/todo-api:1.0.0
>> docker push arijbelaid/todo-frontend:1.0.0
6d9af71a343e: Pushed
cc867501d192: Pushed
96b0df5a785e: Pushed
bf93e0cfd814: Pushed
90d47cf54851: Pushed
e7de839a5dea: Pushed
142dbb135efb: Pushed
c669c4134bf4: Pushed
b0aec4a98ced: Pushed
1.0.0: digest: sha256:879387633c861d16834cdf665bd0696d90053a0e5b37f8ba3d90ad9b740ea017 size: 856
The push refers to repository [docker.io/arijbelaid/todo-frontend]
9553d7f5ad1e: Pushed
416080897f71: Pushed
0acd48a9257f: Pushed
ac634bb45558: Pushed
68312d2f4d4a: Pushed
4abcf2066143: Mounted from testcontainers/ryuk
581d1dfd5a42: Pushed
e7f41c94aec9: Pushed
d036e44b3bd9: Pushed
6052b40e7a94: Pushed
c56a6fd7b6fa: Pushed
479713f85b44: Pushed
1.0.0: digest: sha256:e0d442cc1173a499d2bb15dba58f0eddc8bc7785abe521d0a707d9f37a9a951a size: 856
PS C:\Users\arijb\Desktop\miniproj2>
```

PHASE_2 : Namespace, quotas et limites

1. Créer le fichier quota.yaml



2.Appliquer depuis PowerShell :

```
PS C:\Users\arijb> kubectl create namespace miniproj2
namespace/miniproj2 created
PS C:\Users\arijb> kubectl config set-context --current --namespace=miniproj2
Context "default" modified.
PS C:\Users\arijb> kubectl apply -f .\k8s\00-namespace\quota.yaml
error: the path ".\\k8s\\00-namespace\\quota.yaml" does not exist
PS C:\Users\arijb> cd C:\Users\arijb\Desktop\miniproj2
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\00-namespace\quota.yaml
resourcequota/app-quota created
limitrange/app-limitrange created
PS C:\Users\arijb\Desktop\miniproj2>
```

3. Vérifier :

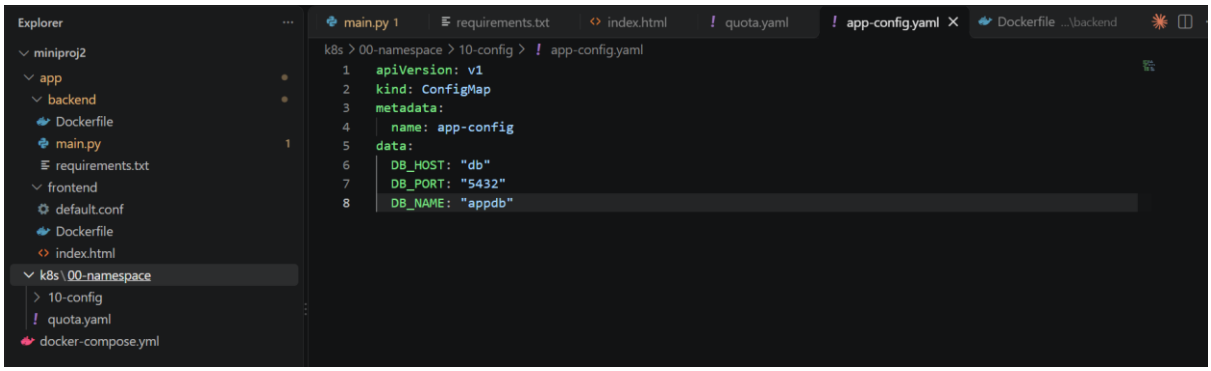
```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get resourcequota
NAME        REQUEST                                                                                 LIMIT
    AGE
app-quota   persistentvolumeclaims: 0/3, pods: 0/10, requests.cpu: 0/1, requests.memory: 0/1Gi   limits.cpu: 0/2, limits.memory: 0/2G
i   36s
PS C:\Users\arijb\Desktop\miniproj2> kubectl describe quota app-quota
Name:                   app-quota
Namespace:              miniproj2
Resource                Used  Hard
--------                ----  ----
limits.cpu              0     2
limits.memory           0     2Gi
persistentvolumeclaims  0     3
pods                    0     10
requests.cpu            0     1
requests.memory         0     1Gi
PS C:\Users\arijb\Desktop\miniproj2>
```

3.test du LimitRange :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl run test-limite --image=nginx:1.26-alpine
pod/test-limite created
PS C:\Users\arijb\Desktop\miniproj2> kubectl describe pod test-limite | findstr /C:"Limits" /C:"Requests"
    Limits:
    Requests:
PS C:\Users\arijb\Desktop\miniproj2> kubectl delete pod test-limite
pod "test-limite" deleted from miniproj2 namespace
PS C:\Users\arijb\Desktop\miniproj2>
```

PHASE_ 3 : ConfigMap et Secret

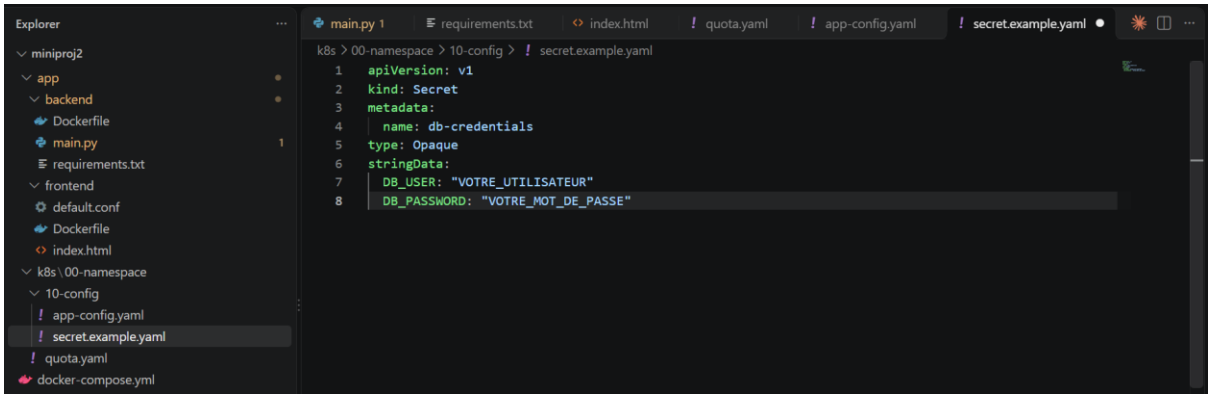
1. Créer le ConfigMap :



2. Créer le Secret

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl create secret generic db-credentials --from-literal=DB_USER=appuser --from-literal=DB_PA
SSWORD=ChangeMe-123
secret/db-credentials created
PS C:\Users\arijb\Desktop\miniproj2>
```

3.Créer le fichier "modèle" pour Git



4. Appliquer le ConfigMap

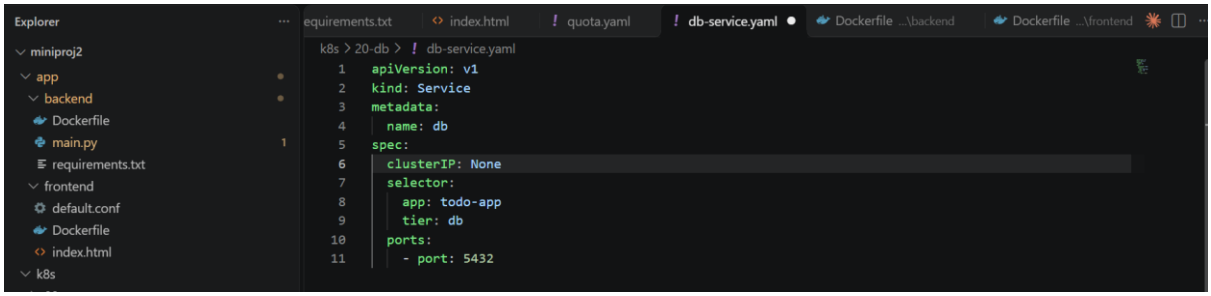
```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\10-config\app-config.yaml
configmap/app-config created
PS C:\Users\arijb\Desktop\miniproj2>
```

5. Vérifier

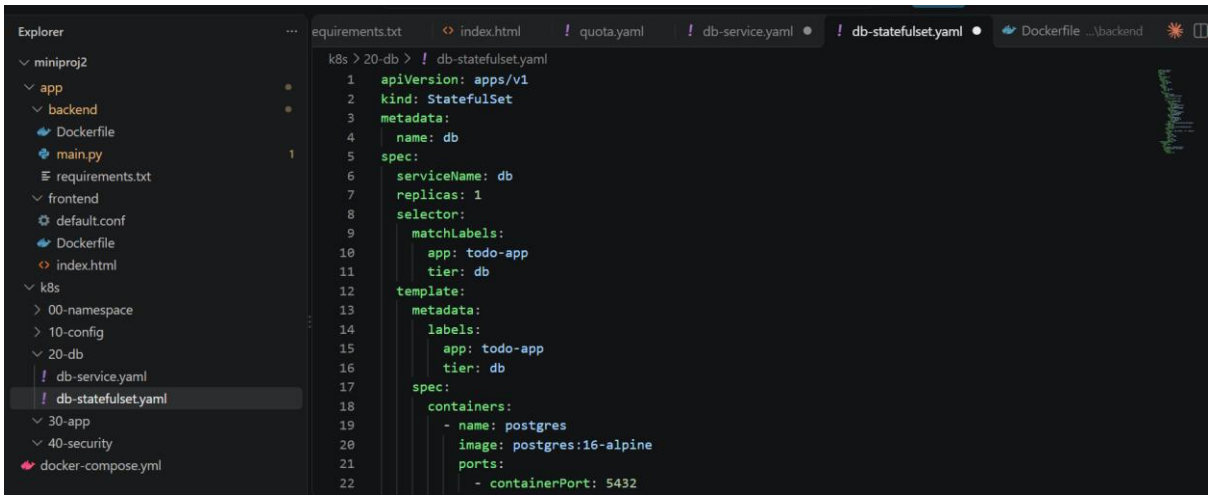
```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get configmap
NAME               DATA   AGE
app-config         3      53s
kube-root-ca.crt   1      16m
PS C:\Users\arijb\Desktop\miniproj2> kubectl get secret
NAME             TYPE     DATA   AGE
db-credentials   Opaque   2      8m22s
PS C:\Users\arijb\Desktop\miniproj2>
PS C:\Users\arijb\Desktop\miniproj2>
```

PHASE_ 4 : Base de données en StatefulSet :

1. Le Service Headless



2. Le StatefulSet :



3. Appliquer :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\20-db\db-service.yaml
service/db created
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\20-db\db-statefulset.yaml
statefulset.apps/db created
PS C:\Users\arijb\Desktop\miniproj2>
```

4.Vérifier :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get pods -w
NAME   READY   STATUS    RESTARTS   AGE
db-0   1/1     Running   0          31s
PS C:\Users\arijb\Desktop\miniproj2>
```

PHASE_ 5 : API et frontend :

1.Créer les fichier ,Appliquer et Vérifier :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\30-app\
deployment.apps/api created
service/api-svc created
configmap/frontend-nginx-config created
deployment.apps/frontend created
service/frontend-svc created
```

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get pods -w
NAME                        READY   STATUS    RESTARTS   AGE
api-698bdb9bcd-9x7d8        1/1     Running   0          2m30s
api-698bdb9bcd-f92vh        1/1     Running   0          2m30s
db-0                        1/1     Running   0          8m40s
frontend-7dddd44bf6-td9ks   1/1     Running   0          2m30s
```

2. Vérifier les logs de l'API :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl logs -l tier=api
Defaulted container "api" out of: api, attendre-db (init)
Defaulted container "api" out of: api, attendre-db (init)
INFO:     10.42.3.1:45364 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.3.1:45366 - "GET /health HTTP/1.1" 200 OK
INFO:     10.42.3.1:45372 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.3.1:36430 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.3.1:36446 - "GET /health HTTP/1.1" 200 OK
INFO:     10.42.3.1:36452 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.3.1:60056 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.3.1:60072 - "GET /health HTTP/1.1" 200 OK
INFO:     10.42.3.1:60084 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.3.1:57064 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:40940 - "GET /health HTTP/1.1" 200 OK
INFO:     10.42.1.1:40946 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:40952 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:42168 - "GET /health HTTP/1.1" 200 OK
INFO:     10.42.1.1:42180 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:42188 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:49382 - "GET /health HTTP/1.1" 200 OK
INFO:     10.42.1.1:49390 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:49400 - "GET /ready HTTP/1.1" 200 OK
INFO:     10.42.1.1:56696 - "GET /health HTTP/1.1" 200 OK
```

PHASE_ 6 : Ingress

1.Créer fichier ingres :

```
PS C:\Users\arijb\Desktop\miniproj2>
PS C:\Users\arijb\Desktop\miniproj2> @'
>> apiVersion: networking.k8s.io/v1
>> kind: Ingress
>> metadata:
>>   name: app-ingress
>> spec:
>>   ingressClassName: traefik
>>   rules:
>>     - host: app.k3s.local
>>       http:
>>         paths:
>>           - path: /
>>             pathType: Prefix
>>             backend:
>>               service:
>>                 name: frontend-svc
>>                 port:
>>                   number: 80
>> '@ | Set-Content -Path .\k8s\30-app\ingress.yaml
PS C:\Users\arijb\Desktop\miniproj2>
```

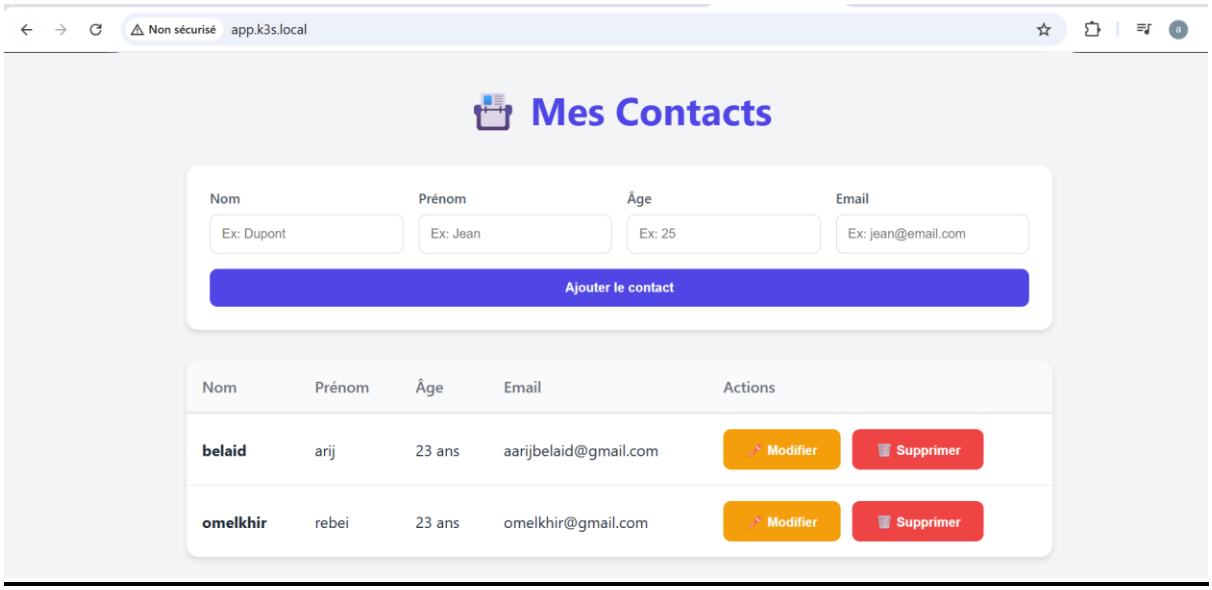
2. Appliquer l'Ingress :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\30-app\ingress.yaml
ingress.networking.k8s.io/app-ingress created
PS C:\Users\arijb\Desktop\miniproj2> kubectl get ingress
NAME          CLASS     HOSTS           ADDRESS                                          PORTS   AGE
app-ingress   traefik   app.k3s.local   192.168.159.10,192.168.159.11,192.168.159.12   80      6s
PS C:\Users\arijb\Desktop\miniproj2>
```

3.Test final :

```
PS C:\Users\arijb\Desktop\miniproj2> curl.exe -H "Host: app.k3s.local" http://192.168.159.10
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gestionnaire de Contacts</title>
    <style>
        :root {
            --primary: #4f46e5;
            --primary-hover: #4338ca;
            --bg: #f3f4f6;
            --card-bg: #ffffff;
            --text: #1f2937;
            --danger: #ef4444;
            --success: #10b981;
        }
        * { box-sizing: border-box; margin: 0; padding: 0; }
        body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background-color: var(--bg); color: var(--text); padding
: 2rem; }

        .container { max-width: 900px; margin: 0 auto; }
        h1 { text-align: center; margin-bottom: 2rem; color: var(--primary); font-size: 2.5rem; }
```



PHASE_ 7 : Persistance et résilience

1.ajouter un contact et vérifier dans bdd :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl exec -it db-0 -- psql -U appuser -d appdb -c "SELECT * FROM contacts;"
 id |   nom    | prenom | age |         email
----+----------+--------+-----+-----------------------
  1 | belaid   | arij   |  23 | aarijbelaid@gmail.com
  2 | omelkhir | rebei  |  23 | omelkhir@gmail.com
  3 | Dupont   | Jean   |  30 | jean@test.com
(3 rows)
```

2. Détruire le Pod de la base de données

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl delete pod db-0
pod "db-0" deleted from miniproj2 namespace
PS C:\Users\arijb\Desktop\miniproj2>
```

3. Vérifier la recréation :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get pods -w
NAME                        READY   STATUS    RESTARTS   AGE
api-698bdb9bcd-9x7d8        1/1     Running   0          26m
api-698bdb9bcd-f92vh        1/1     Running   0          26m
db-0                        1/1     Running   0          26s
frontend-7dddd44bf6-td9ks   1/1     Running   0          26m
```

4.Vérifier dans bdd :

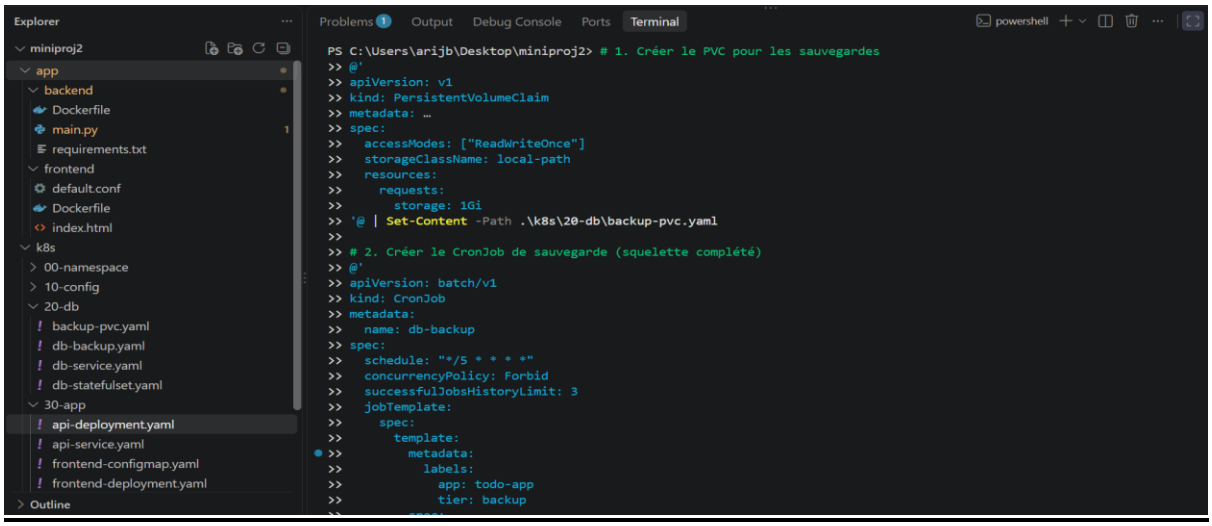
```
PS C:\Users\arijb\Desktop\miniproj2> kubectl exec -it db-0 -- psql -U appuser -d appdb -c "SELECT * FROM contacts;"
 id |   nom    | prenom | age |         email
----+----------+--------+-----+-----------------------
  1 | belaid   | arij   |  23 | aarijbelaid@gmail.com
  2 | omelkhir | rebei  |  23 | omelkhir@gmail.com
  3 | Dupont   | Jean   |  30 | jean@test.com
(3 rows)
```

5. Résilience de l'API

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl delete pod -l tier=api
pod "api-698bdb9bcd-9x7d8" deleted from miniproj2 namespace
pod "api-698bdb9bcd-f92vh" deleted from miniproj2 namespace
PS C:\Users\arijb\Desktop\miniproj2>
```

PHASE_ 8 : Sauvegarde planifiée et restauration

1.Créer le PVC de sauvegarde et le CronJob



2. Appliquer

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\20-db\backup-pvc.yaml
persistentvolumeclaim/backup-pvc created
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\20-db\db-backup.yaml
cronjob.batch/db-backup created
PS C:\Users\arijb\Desktop\miniproj2> kubectl create job --from=cronjob/db-backup test-backup
job.batch/test-backup created
PS C:\Users\arijb\Desktop\miniproj2>
```

3. Vérifier

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get pods
NAME                         READY   STATUS      RESTARTS   AGE
api-698bdb9bcd-g9rdb         1/1     Running     0          5m36s
api-698bdb9bcd-q8mb2         1/1     Running     0          5m37s
db-0                         1/1     Running     0          7m44s
db-backup-29882795-qqxx8     0/1     Completed   0          54s
frontend-7dddd44bf6-td9ks    1/1     Running     0          33m
test-backup-xtjms            0/1     Completed   0          116s
PS C:\Users\arijb\Desktop\miniproj2>
```

4.Véfirier bdd :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl exec -it db-0 -- psql -U appuser -d appdb -c "SELECT * FROM contacts;"
 id |   nom    | prenom | age |         email
----+----------+--------+-----+-----------------------
  1 | belaid   | arij   |  23 | aarijbelaid@gmail.com
  2 | omelkhir | rebei  |  23 | omelkhir@gmail.com
  4 | Dupont   | Prénom |  30 | jean@test.com
(3 rows)
```

PHASE 9 : RBAC :

1. Créer les ServiceAccounts

```
PS C:\Users\arijb\Desktop\miniproj2> @'
>> apiVersion: v1
>> kind: ServiceAccount
>> metadata:
>>   name: api-sa
>> automountServiceAccountToken: false
>> ---
>> apiVersion: v1
>> kind: ServiceAccount
>> metadata:
>>   name: frontend-sa
>> automountServiceAccountToken: false
>> ---
>> apiVersion: v1
>> kind: ServiceAccount
>> metadata:
>>   name: db-sa
>> automountServiceAccountToken: false
>> ---
>> apiVersion: v1
>> kind: ServiceAccount
>> metadata:
>>   name: dev-viewer
>> '@ | Set-Content -Path .\k8s\40-security\serviceaccounts.yaml
```

2. Créer le Role et le RoleBinding

```
>> '@ | Set-Content -Path .\k8s\20-db\restore.yaml
PS C:\Users\arijb\Desktop\miniproj2> @'
>> apiVersion: v1
>> kind: ServiceAccount
>> metadata:                          @'
>> apiVersion: rbac.authorization.k8s.io/v1
>> kind: RoleerviceAccountToken: false
>> metadata:
>>   name: pod-reader
>> rules:ServiceAccount
>>   - apiGroups: [""]
>>     resources: ["pods", "pods/log", "services"]
>>     verbs: ["get", "list", "watch"]
>> ---
>> apiVersion: rbac.authorization.k8s.io/v1
>> kind: RoleBindingunt
>> metadata:
>>   name: dev-viewer-binding
>> subjects:ServiceAccountToken: false
>>   - kind: ServiceAccount
>>     name: dev-viewer
>> roleRef:rviceAccount
>>   kind: Role
>>   name: pod-reader
>>   apiGroup: rbac.authorization.k8s.iority\serviceaccounts.yaml
>> '@ | Set-Content -Path .\k8s\40-security\rbac.yaml
```

3. Appliquer les configurations

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\40-security\serviceaccounts.yaml
serviceaccount/api-sa created
serviceaccount/frontend-sa created
serviceaccount/db-sa created
serviceaccount/dev-viewer created
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\40-security\rbac.yaml
role.rbac.authorization.k8s.io/pod-reader created
rolebinding.rbac.authorization.k8s.io/dev-viewer-binding created
PS C:\Users\arijb\Desktop\miniproj2>
```

4. Vérification

```
PS C:\Users\arijb\Desktop\miniproj2> $SA = "system:serviceaccount:miniproj2:dev-viewer"
PS C:\Users\arijb\Desktop\miniproj2>
PS C:\Users\arijb\Desktop\miniproj2> # 1. Lister les pods (Attendu : yes)
PS C:\Users\arijb\Desktop\miniproj2> kubectl auth can-i list pods --as=$SA
yes
PS C:\Users\arijb\Desktop\miniproj2>
PS C:\Users\arijb\Desktop\miniproj2> # 2. Lire les logs (Attendu : yes)
PS C:\Users\arijb\Desktop\miniproj2> kubectl auth can-i get pods/log --as=$SA
yes
PS C:\Users\arijb\Desktop\miniproj2>
PS C:\Users\arijb\Desktop\miniproj2> # 3. Supprimer un pod (Attendu : no)
PS C:\Users\arijb\Desktop\miniproj2> kubectl auth can-i delete pods --as=$SA
no
PS C:\Users\arijb\Desktop\miniproj2>
PS C:\Users\arijb\Desktop\miniproj2> # 4. Lire les secrets (Attendu : no)
PS C:\Users\arijb\Desktop\miniproj2> kubectl auth can-i get secrets --as=$SA
no
PS C:\Users\arijb\Desktop\miniproj2>
```

5.modifier api-deployment.yaml , frontend-deployment.yaml et db-statefulset.yaml

6. Appliquer les modifications au cluster

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\30-app\api-deployment.yaml
deployment.apps/api configured
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\30-app\frontend-deployment.yaml
deployment.apps/frontend configured
PS C:\Users\arijb\Desktop\miniproj2> kubectl apply -f .\k8s\20-db\db-statefulset.yaml
statefulset.apps/db configured
PS C:\Users\arijb\Desktop\miniproj2>
```

7. Le test ultime

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl exec -it api-xxxxx -- ls /var/run/secrets/kubernetes.io/serviceaccount
Error from server (NotFound): pods "api-xxxxx" not found
PS C:\Users\arijb\Desktop\miniproj2>
```

PHASE_10 : Durcissement et NetworkPolicies :

1. Preuve de la désactivation du montage du jeton de ServiceAccount (E12) :

```
PS C:\Users\arijb\Desktop\miniproj2> $API_POD = kubectl get pods -l tier=api -o jsonpath="{.items[0].metadata.name}"
PS C:\Users\arijb\Desktop\miniproj2> kubectl exec -it $API_POD -- ls /var/run/secrets/kubernetes.io/serviceaccount
Defaulted container "api" out of: api, attendre-db (init)
ls: cannot access '/var/run/secrets/kubernetes.io/serviceaccount': No such file or directory
command terminated with exit code 2
PS C:\Users\arijb\Desktop\miniproj2>
```

2. Test de la NetworkPolicy :

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl run pirate --rm -it --image=busybox:1.36 --restart=Never -- wget -T 3 -qO- http://db:543
2
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed
through the command prompt.
If you don't see a command prompt, try pressing enter.
pod "pirate" deleted from miniproj2 namespace
pod miniproj2/pirate terminated (Error)
PS C:\Users\arijb\Desktop\miniproj2>
```

3.Vérification du SecurityContext (runAsNonRoot, drop ALL, readOnlyRootFilesystem)

```
PS C:\Users\arijb\Desktop\miniproj2> kubectl get pod $API_POD -o jsonpath="{.spec.containers[0].securityContext}"
{"allowPrivilegeEscalation":false,"capabilities":{"drop":["ALL"]},"readOnlyRootFilesystem":true}
PS C:\Users\arijb\Desktop\miniproj2>
```

4.Validation du fonctionnement de l'application 3-tiers malgré les restrictions réseau.

